

Roteiro de Apresentação: Sistema de Gerenciamento de Estudantes

Duração Estimada: 9 a 11 minutos

Formato: Apresentação sequencial com foco na arquitetura e detalhes técnicos do código-fonte.

1. Introdução e Visão Geral (00:00 - 01:30)

- **Abertura:** "Olá a todos, hoje vou apresentar nosso Projeto de Extensão: um Sistema de Gerenciamento de Estudantes desenvolvido inteiramente em Java puro."
- **Contextualização:** "O objetivo do projeto é aplicar os conceitos de programação orientada a objetos (POO), e integração com banco de dados relacional. Para isso, escolhemos o **PostgreSQL**."
- **Arquitetura do Projeto:** "Estruturamos o código em pacotes que imitam o padrão MVC, para garantir um baixo acoplamento e alta coesão."
 - *Mencionar os pacotes: model (entidades), data (persistência), view (interface de console) e main (inicialização).*

2. Modelagem de Dados: A Base do Sistema (01:30 - 02:30)

- **Foco:** `src/model/Estudante.java`
- **Fala Sugerida:** "Começando pela fundação do nosso sistema, temos o pacote `model` contendo a classe `Estudante`. Ela é um POJO simples que representa a nossa entidade de negócio principal. Possui apenas dois atributos privados básicos: o `nome` e o `curso`. Empregamos o encapsulamento através de construtores padronizados e métodos de acesso (Getters e Setters) para proteger a integridade dos dados na memória antes de irem para o banco."

3. Conexão e Preparação do Banco de Dados (02:30 - 04:30)

- **Foco:** `ConexaoBancoDeDados.java` e `CriadorDeTabelas.java` (Pacote `data`)
- **Conexão:** "Para conectar o sistema ao banco, criamos a classe `ConexaoBancoDeDados`. Ela utiliza a biblioteca JDBC. Temos um método estático `obterConexao()` que configura a URL JDBC apontando para o banco local 'Escola', insere as credenciais do usuário 'postgres' e utiliza o `DriverManager` para estabelecer essa ponte de dados. Tratamos os erros com um bloco `try-catch` para que o sistema não pare de forma abrupta caso o banco esteja fora do ar."
- **Criação Automática:** "Para facilitar a instalação do software em qualquer máquina, implementamos a classe `CriadorDeTabelas`. Ela é responsável por pegar uma string SQL que diz `CREATE TABLE IF NOT EXISTS estudantes` e rodar essa query via objeto `Statement`. Isso garante que as tabelas necessárias existirão antes do usuário tentar inserir qualquer dado."

4. O Coração do Sistema: Operações CRUD (04:30 - 08:00)

- **Foco:** `BancoDeDados.java` (Pacote `data`)
- **Introdução da Classe:** "A classe mais robusta do projeto é a `BancoDeDados.java`. Ela recebe, via injeção de dependência no construtor, a conexão ativa e o scanner. É ela que executa o CRUD."
- **Insert (Adição):** "No método `adicionarEstudante`, coletamos os dados e instanciamos um objeto `Estudante`. Depois, criamos dinamicamente uma instrução SQL `INSERT INTO`. Usamos o `stmt.executeUpdate()` para salvar."
- **Update (Edição):** "Temos três cenários de edição: editar só o nome, só o curso ou ambos. Para todos eles, aplicamos comandos `UPDATE` utilizando a cláusula `WHERE`. O interessante aqui é que validamos o retorno do banco. Se a query retornar zero linhas afetadas, avisamos o usuário que o estudante não foi encontrado."
- **Delete (Remoção):** "Para as exclusões, permitimos flexibilidade. O usuário pode deletar através de uma chave primária (ID), através do nome exato, ou até mesmo realizar uma deleção em lote utilizando o método `removerEstudantesCurso()`, que apaga todos os estudantes vinculados a um curso específico."
- **Read (Leituras):** "Finalizando as operações, os métodos de listagem utilizam a interface `ResultSet`. Ao invés de um update, fazemos um `stmt.executeQuery()`. Percorremos o resultado utilizando um loop `do-while` junto com `rs.next()`, imprimindo de forma tabelada o ID, Nome e Curso."

5. Interface com o Usuário e Roteamento (08:00 - 09:30)

- **Foco:** `Menu.java` (Pacote `view`)
- **Funcionamento:** "Para tornar o software interativo, criamos a classe `Menu`. Ela possui um loop `while` que mantém o programa ativo. Desenvolvemos uma interface via terminal amigável (CLI) exibindo as 5 operações principais."
- **Tratamento de Erros:** "Um ponto forte do menu é a robustez. Implementamos um tratamento de exceções (try-catch) focado em evitar travamentos. Se o sistema espera um número de menu, e o usuário digita uma letra, capturamos a exceção de input, limpamos o buffer do scanner e pedimos a opção novamente."

- **Submenus:** "Utilizamos um bloco `switch-case` principal. Para opções complexas como Edição, Remoção e Listagem, apresentamos sub-menus antes de invocar diretamente os métodos da classe `BancoDeDados`."

6. O Orquestrador: Main e Considerações Finais (09:30 - 11:00)

- **Foco:** `SistemaGerenciamentoEstudantes.java` (Pacote `main`)
- **Ponto de Partida:** "Finalmente, a classe principal que amarra tudo. No `main()`, nós primeiro instanciamos as dependências críticas (conexão e scanner), passamos essas dependências para os objetos que precisam delas (como o BD e o Menu)."
- **Desligamento Seguro:** "Após o usuário escolher a opção de Sair no menu, o código retoma o controle na `main` e passamos por um bloco final para realizar o fechamento seguro do banco: `conexao.close()`. Isso evita vazamento de memória e conexões presas no PostgreSQL. Depois disso, forçamos o encerramento com `System.exit(0)`."
- **Considerações Finais:** "Como conclusão, este projeto consolidou de forma prática o entendimento sobre persistência de dados em Java, injeção de dependência básica e o uso de JDBC (Java Database Connectivity) sem frameworks, o que permitiu o entendimento da mecânica de baixo nível de bancos de dados. A separação por pacotes nos deixa prontos para uma eventual migração para uma interface gráfica no futuro."
- **Encerramento:** "Agradeço a atenção de todos e abro agora o espaço para eventuais perguntas."